
EDA Bench: An Execution-Based Benchmark for Language Model Agents on Functional PCB Construction

Dhruv Saini
Independent Researcher
dhruv9saini@gmail.com

Abstract

Language-model agents increasingly operate electronic design automation tools: they search datasheets, select components, edit KiCad projects, run design checks, and iterate after failures. Reliable PCB construction requires correct interfaces, routed connectivity, electrical behavior, and manufacturable geometry. EDA Bench is an execution-based benchmark with 35 difficult KiCad PCB construction tasks derived from public open-hardware projects, spanning connector breakouts, power-path circuits, microcontroller systems, motor drivers, camera and compute adapters, and high-density carriers. Each harness receives a detailed engineering brief and a standard KiCad environment and produces a complete KiCad project. The behavioral evaluator parses the PCB, extracts routed copper, checks declared external ports, models trace parasitics and short faults, runs ngspice simulations, and applies KiCad ERC and DRC checks. External-I/O behavior determines the score, while reference, failure, and mutation controls calibrate it: all 35 frozen references score 1.0, and all 35 structural-failure controls score at most 0.15. The strongest evaluated web-enabled harness scores 4.58% overall and builds 21 of 35 projects. These results establish complete, electrically correct PCB construction as an extremely difficult task for current agent systems.

1 Introduction

Language-model agents can now operate development tools over long horizons. Execution-based benchmarks measure the resulting artifacts in the environments where they run: an agent modifies an artifact, executes tools, observes failures, and iterates. SWE-bench evaluates agents on real GitHub issues whose solutions must pass repository tests [Jimenez et al., 2023]. OSWorld extends execution-based evaluation to open-ended desktop and web tasks in real computer environments [Xie et al., 2024]. ProgramBench evaluates construction from an executable and documentation, using behavioral tests to compare the rebuilt program with the reference [Yang et al., 2026]. Together, these benchmarks establish behavior in an execution environment as a strong measure of artifact quality.

Hardware design has an analogous workflow. Engineers search for parts, inspect datasheets, draft schematics, route boards, run ERC and DRC checks, simulate where possible, and produce manufacturing files. EDA evaluation must cover routed connectivity, electrical behavior, manufacturability constraints, and external interfaces. Existing CAD and EDA benchmarks address text-to-CAD generation [Khan et al., 2024], geometric CAD validation [Epoch AI, 2026, CAD Arena, 2026, Barkley et al., 2026], schematic generation [Al Hasan et al., 2026, Zou et al., 2026, Luo et al., 2026], schematic understanding [Lu et al., 2026], board-level schematic design [Qiu et al., 2026], hardware bug repair [Cui et al., 2026], and PCB placement and routing reasoning [Li et al., 2026]. EDA Bench extends this landscape to complete routed KiCad projects whose external electrical behavior is checked directly.

Table 1: EDA Bench evaluation pipeline. Project parsing, electrical checks, and simulation score each complete harness output.

Stage	Artifact or operation
Task input	The harness receives a fixed engineering brief, runtime task pack, tool environment, model configuration, and access policy.
Harness execution	The agent operates in the Dockerized KiCad environment and writes a project to <code>/workspace/final_project</code> .
Project parsing	The grader locates KiCad files and extracts connectors, pads, nets, board outline, and routed copper geometry.
Contract binding	External ports from the submitted board are matched to task-declared I/O contracts and expected measurements.
Execution checks	The grader applies KiCad ERC and DRC diagnostics, ngspice simulations, routed-I/O checks, parasitic models, and short-fault checks.
Score reporting	The evaluator returns a normalized task score, hard-cap diagnostics, build status, and run artifacts.

EDA Bench evaluates complete agent harnesses on source-backed KiCad PCB construction tasks. A harness receives a detailed, self-contained engineering brief, then writes a complete KiCad project in `/workspace/final_project`. KiCad provides an open-source EDA environment for schematic capture, integrated simulation, PCB layout, 3D rendering, and manufacturing export [KiCad Project, 2026]. Each brief specifies the board purpose, external interfaces, power behavior, approximate form factor, layer count, core components, and completion requirements. Web-enabled harnesses may consult datasheets and tool documentation while carrying out the design.

The scoring system measures observable behavior at the realized board’s external electrical interfaces. It parses, checks, and simulates each project against a task contract, with ngspice as the open-source circuit-simulation backend [ngspice contributors, 2026]. Different internal implementations can earn the same score when they expose the required behavior. Missing, shorted, swapped, isolated, or weakly driven external nets reduce the score through direct electrical-interface checks. Frozen references, structural-failure controls, mutation controls, and representation-invariance checks establish the evaluator’s sensitivity to engineering-relevant PCB changes.

2 Contributions

EDA Bench provides 35 complete-board construction tasks ranging from compact breakouts to eight-layer compute modules; self-contained engineering briefs with electrical, interface, component, and physical requirements; an execution-based evaluator that combines routed-copper analysis, external-I/O simulation, and KiCad design checks; and calibrated controls that establish score sensitivity across valid designs and common electrical failures. The released harness, task packs, run records, and grading artifacts support repeatable evaluation and detailed failure analysis.

3 Benchmark overview

EDA Bench evaluates PCB construction as an execution task. For each task, a harness receives a fixed engineering brief, runtime task pack, tool environment, model configuration, and access policy, then produces a complete KiCad project in `/workspace/final_project`. The automated evaluator parses the KiCad files, extracts external connectors and routed copper, applies task-declared stimuli and probes, runs simulations, checks ERC and DRC diagnostics, and assigns a normalized score based on the realized board behavior.

Table 1 summarizes the execution and grading pipeline. Each released task is derived from a public hardware project and has a frozen reference design. The evaluated runtime pack contains a self-contained engineering brief and task metadata, while the grading pack contains the reference project, source snapshot, controls, contracts, and evaluator implementation. This arrangement gives the agent a complete construction specification and keeps grading inputs consistent across harnesses.

Scores characterize complete evaluated harnesses. A harness may include prompting, web search for technical documentation, KiCad tool use, repair loops, and project-generation logic. The protocol

Table 2: Task distribution and representative categories in EDA Bench.

Difficulty	Tasks	Representative task families
Very easy	2	Compact connector breakouts and small adapters
Easy	5	I/O breakouts, logic-level conversion, USB/UART basics
Medium	4	Power-path boards, serial transceivers, small controller boards
Hard	12	Compute adapters, sensor/camera boards, multi-connector carriers
Very hard	7	Dense mixed-signal boards and larger interface carriers
Extreme	5	Large baseboards and high-density compute boards

evaluates the system architecture as a whole, beyond just a standalone model, because practical EDA automation depends on operating the full workflow.

The score measures declared external-I/O behavior under the released task contracts. Equivalent internal implementations receive credit when they realize the required behavior, while missing, isolated, shorted, swapped, unrouted, or weakly driven external nets trigger corresponding deductions. This execution-based scope complements fabrication, regulatory, thermal, EMC, and vendor-silicon review.

4 Construction tasks

EDA Bench contains 35 source-backed tasks. Each task is anchored to a public upstream hardware project and a frozen reference design. The task set covers six difficulty levels, shown in Table 2.

Each released task requires a source design, a frozen KiCad reference, separate runtime and grading packs, an external-I/O contract, control submissions, license triage, and evaluator support. The resulting 35-task set prioritizes difficult, deeply checked board construction across six difficulty levels.

The task catalog includes compact breakouts and converters such as `usb_c_female_breakout`, `mcp23017_breakout`, and `rs485_transceiver_breakout`. It also includes power and control boards such as `bq24295_power_path_board`, `picopd`, and `robotont_driver_board`, compute and camera or display adapters such as `cm4_csi_adapter`, `m2_pcie_adapter`, and `ov9281_dual_camera_board`, and large carrier boards such as `cm4_baseboard`, `jetson_orin_baseboard`, and `zynq_som`.

Each task has two artifact forms. The runtime task pack is mounted for the evaluated agent and contains the prompt, task metadata, and runtime assets. The public grading pack contains the reference project, upstream snapshot, control submissions, license notes, `io_contract.json`, `functional_contract.json`, and local tests. Separate packs preserve a clean task boundary and support exact reproduction of grading results.

5 Harness protocol

A harness is evaluated as a complete system. It receives the task prompt and must produce a KiCad project in `/workspace/final_project`. Built-in harnesses run inside Docker with KiCad, ngspice, Python, Node, internet access, and the configured model interface. The evaluated harnesses are Codex CLI with GPT-5.5 web search at low, medium, high, and xhigh reasoning, Pi with Gemini 3.1 Pro Preview web high, and Pi with DeepSeek V4 Pro web high.

This protocol evaluates the system architecture as a whole, beyond just a standalone model. A one-shot generator, a tool-using agent, and a multi-agent harness can all target the same task interface. EDA work is iterative: the system creates files, runs KiCad tools, inspects failures, consults technical documentation, and repairs invalid output.

Each reported run includes task prompts, command lines, usage summaries, grader outputs, final projects, and generated artifacts. These records allow final projects to be regraded without rerunning model harnesses.

Table 3: Representative task-family checks layered on the generic external-I/O evaluator. These checks bind measured signals to task contracts using deterministic circuit and channel models.

Task family	Declared behavior	Common hard caps
Connector breakouts and simple adapters	Connector pins and power/ground pads, continuity, shorts, routed coverage, pin/net semantic agreement	Swapped pins, split nets, isolated pads, power-ground shorts
Serial and level-shifting interfaces	USB/UART/RS-485/logic-side boundary ports, signal transfer families, active-device realization	Missing active path, weak external mapping, missing routed I/O
Power-path and charger boards	Input, battery/load, configuration, and status ports, power path presence, external safety	Missing active power role, shorts, missing component-function profile
Camera/display/compute carriers	High-density connector families and differential pairs, routed families, pair plausibility	Missing high-speed pair families, weak routing, shorts, split nets
Large mixed-signal baseboards	Multi-rail power, GPIO, debug, serial, high-speed, and peripheral interfaces	Incomplete connector semantics, missing active realization, shorts

6 Scoring system

The scorer evaluates a submitted KiCad project through explicit I/O simulation. Unsupported, missing, or malformed submissions receive zero score. Supported tasks declare their external ports, stimuli, probes, simulator models, expected measurements, and tolerances. For each submission, the evaluator locates and parses the KiCad project, runs ERC and DRC diagnostics, extracts connectors, pads, nets, and routed copper geometry, binds submitted external ports to the task contract, constructs task-declared probes and stimuli, computes task-family measurements and hard caps, and returns a normalized score with diagnostic artifacts.

The primary score is a normalized task score in $[0, 1]$. A benchmark aggregate is a per-task difficulty-weighted mean:

$$S = \frac{\sum_{i=1}^N w(d_i) s_i}{\sum_{i=1}^N w(d_i)}, \quad w(d_i) \in \{1.0, 1.5, 2.0, 3.0, 4.0, 5.0\}. \quad (1)$$

We also report the unweighted mean over tasks. Build success means that the submission produced a KiCad project that the grader could parse and evaluate. It is not equivalent to functional success.

The main score combines three observable properties of the submitted PCB: external connector semantics, routed-copper behavior, and task-required active and manufacturing plausibility. The evaluator applies deterministic ngspice stimuli at declared external ports, samples boundary waveforms, and caps scores for missing routed copper, swapped external pins, shorts, missing active circuitry, unpowered active devices, invalid high-speed pair families, and gross fabrication implausibility. Reference designators, component inventory, footprint similarity, outline similarity, and route-shape similarity provide supporting diagnostics.

Within each task, declared measurements produce normalized sub-scores for external-port mapping, routed continuity, simulation behavior, and task-family realization checks. These sub-scores are combined according to the task contract to form a soft functional score. Hard caps then impose upper bounds for structural failures that make the board externally incorrect or physically implausible, such as shorts, missing routed I/O, swapped connector pins, or absent active circuitry. The final task score is the resulting capped functional score. This design allows partial credit for partially correct external behavior while preventing submissions with severe structural faults from receiving high scores through unrelated checks.

Table 4: Evaluator-validity evidence. The controls calibrate the released score as an external-I/O contract measure.

Evidence	What it supports
35/35 released frozen human-designed reference PCBs score 1.0	The tasks are solvable, and real upstream designs satisfy their declared contracts.
35/35 released structural fail canaries score at most 0.15	The score is not reducible to file existence, KiCad syntax, or component-shell matching.
162 mutation canaries pass validation	The evaluator is sensitive to shorts, unrouted nets, isolated pads, missing outlines, and missing high-speed-like routing.
Refdes-renamed USB-C board scores 1.0	Renaming connector reference designators preserves the score.
Plausible USB-C route-width edit remains at least 0.85	Plausible route-width changes preserve high scores.
Tiny-trace USB-C control scores at most 0.35	Manufacturability caps affect scores for physically implausible routing.

Table 5: Complete web-enabled harness baselines. The primary score is difficulty-weighted, and the unweighted score is the mean task score.

Harness	Weighted	Unweighted	Builds
Codex GPT-5.5 web low	0.0124	0.0143	31/35
Codex GPT-5.5 web medium	0.0279	0.0314	35/35
Codex GPT-5.5 web high	0.0293	0.0229	11/35
Codex GPT-5.5 web xhigh	0.0458	0.0500	21/35
Pi Gemini 3.1 Pro Preview web high	0.0113	0.0086	1/35
Pi DeepSeek V4 Pro web high	0.0000	0.0000	0/35

7 Artifact validation

Because EDA Bench uses task-specific evaluation, the release includes controls that calibrate score interpretation. The frozen reference project serves as the positive control and must score 1.0. The structural-failure control removes active internal circuitry when available or removes the external boundary on passive boards; it must score below the 0.75 release threshold.

Evaluator validity. Score changes should track engineering-relevant changes to declared external-I/O behavior. Positive controls establish that human-designed reference boards are realizable and score 1.0. Structural-failure controls measure sensitivity to function-breaking changes. Invariance controls confirm that benign representation changes, such as reference-designator renaming or plausible route-width edits, have little effect.

A full local sweep on May 6, 2026 verified all 35 released task canaries: every pass canary scored 1.0, and every fail canary scored at most 0.15. The generated-mutation sweep graded 162 mutation canaries across released task families. All tasks passed, with every reference scoring 1.0 and every structural fail canary scoring at most 0.15. Mutations covered missing board outlines, unrouted external nets, isolated external pads, power or signal pads tied to ground, and missing high-speed-like routes for high-speed carrier tasks.

Together, these controls show that the score accepts human-designed reference boards, rejects common function-breaking PCB mutations, and remains stable under several superficial implementation changes.

8 Experiments

We evaluate six web-enabled harness configurations on the 35 released tasks. All runs use the same task prompts, task packs, Docker image, and grading protocol. Table 5 reports a May 6, 2026 no-model regrade of saved final projects under the evaluator revision used for this manuscript.

Complete PCB construction remains extremely difficult. The Codex GPT-5.5 web medium harness builds all 35 released projects and scores 2.79%, while the xhigh harness builds 21 projects and reaches the best score of 4.58%. The large separation between build rate and behavioral score shows

that parseable KiCad output captures only an early stage of the task. Reasoning and tool-use choices also change the distribution of electrical failures.

Difficulty breakdowns further indicate that the benchmark is not saturated by easy tasks. The Codex GPT-5.5 web xhigh harness reaches 13.75% on medium tasks and 7.08% on hard tasks, but only 2.00% on extreme tasks. The gap grows as boards require larger external contracts, active circuitry, dense routing, or high-speed interfaces.

Statistical interpretation. The result table contains one complete released-task sweep per configuration. It supports task-level failure analysis, absolute difficulty estimates, and comparisons between build rate and behavioral score. Repeated independent sweeps would be required for pairwise model-ranking significance tests.

9 Failure analysis

The best evaluated configuration scores 4.58% overall, placing complete PCB construction among the hardest evaluated agent tasks. One harness produces buildable KiCad projects for all 35 tasks yet scores 2.79%, showing that most difficulty lies beyond file generation. Codex GPT-5.5 configurations with different reasoning settings also produce distinct build and score tradeoffs, reinforcing the importance of evaluating the complete system architecture.

Saved grading artifacts expose the main failure modes. Among buildable Codex GPT-5.5 web xhigh submissions, the most common caps were split required external-net continuity, isolated external-I/O pads, wrong external-I/O pin/net mapping, missing or weak component-function realization, missing or implausible high-speed pair geometry, and weak active-device power integrity. The all-building Codex GPT-5.5 web medium run shows a similar pattern: wrong external mapping, missing component-function realization, split required external nets, same-layer trace shorts, and isolated external-I/O pads.

These caps correspond to concrete electrical failures. Submissions often instantiate connector footprints with plausible labels while leaving required I/O pads isolated, splitting required nets across disconnected copper, swapping external pin semantics, omitting active circuitry, or routing high-speed-like interfaces implausibly. The behavioral score measures these difficult board-level requirements directly.

10 Related work

Execution-based software-agent benchmarks. Execution-based software benchmarks evaluate agents by running the artifacts they produce. SWE-bench frames software engineering as repository-level issue resolution, where an agent edits a real codebase and must pass tests associated with the target issue [Jimenez et al., 2023]. OSWorld evaluates multimodal agents in real desktop and web environments with custom execution-based evaluation scripts [Xie et al., 2024]. ProgramBench studies a setting in which agents rebuild software from an executable and documentation, and behavioral tests are generated to compare the candidate program with the reference executable [Yang et al., 2026]. EDA Bench applies this behavioral-evaluation perspective to PCB construction, where correctness depends on realized geometry, external connectivity, and electrical-interface behavior.

CAD generation and geometric evaluation. Text2CAD introduced a large-scale text-to-parametric-CAD dataset and model for generating CAD command sequences from natural-language prompts [Khan et al., 2024]. CadEval and CAD Arena provide evaluation settings for text-to-CAD models using geometric validity, rendering, and prompt-level comparisons [Epoch AI, 2026, CAD Arena, 2026]. CADSmith further emphasizes programmatic geometric validation for generated CadQuery models [Barkley et al., 2026]. These works motivate execution-based evaluation for engineering artifacts. PCB construction also requires connectivity, routing, short avoidance, active-circuit realization, and external-I/O behavior.

Circuit, schematic, and PCB benchmarks. Recent EDA benchmarks study several adjacent tasks. CircuitLM and PCBSchemaGen generate or synthesize circuit schematics from natural-language prompts and use structural or constraint-based checks to improve schematic validity [Al Hasan et al.,

2026, Zou et al., 2026]. SchGen studies PCB schematic generation using semantic-grounded code representations [Luo et al., 2026]. OmniSch evaluates multimodal schematic understanding and schematic-to-graph reasoning over real-world diagrams [Lu et al., 2026]. HWE-Bench evaluates board-level schematic design with datasheet context and hardware bug repair over repository-scale hardware projects [Qiu et al., 2026, Cui et al., 2026]. PCB-Bench evaluates LLM reasoning over placement, routing, and PCB design comprehension using text, images, and complete PCB projects [Li et al., 2026]. EDA Bench differs by requiring an agent to produce a complete routed KiCad PCB project and by scoring the realized board through external-I/O simulation and PCB-geometry checks rather than schematic-only validity, visual reasoning, or placement/routing question answering.

11 Limitations

The released tasks use public hardware designs as grading anchors. Their runtime briefs provide the electrical, interface, component, and physical requirements needed for construction. Scores measure EDA task solving under fixed prompts, tools, task-pack revisions, run records, and access policies.

The 35 released tasks emphasize depth: every task requires a real source design, a reference project, task contracts, controls, license review, and grader support. Additional source-backed packs and a sealed private split would expand coverage across more PCB families.

The behavioral evaluator uses deterministic generic models for external-I/O behavior. Manufacturing review adds electrical safety, regulatory, high-speed protocol, thermal, power-sequencing, and production-readiness analysis. Richer per-task SPICE, IBIS, S-parameter, and field-solver models would extend fidelity beyond the four currently calibrated board families.

Task artifacts inherit mixed upstream hardware licenses. The public artifact includes only score-supported tasks whose runtime and grading packs can be redistributed under the release policy.

12 Conclusion

EDA Bench evaluates complete agent systems on electrically grounded PCB construction. Real public hardware projects, explicit I/O simulation, routed-copper analysis, and KiCad checks turn external-interface behavior into a reproducible score. Current systems can often produce parseable KiCad projects, while the best evaluated harness reaches only 4.58% on the full board-level task. This large gap establishes reliable PCB construction as an extremely difficult target for agentic EDA systems.

References

- Khandakar Shakib Al Hasan, Syed Rifat Raiyan, Hasin Mahtab Alvee, and Wahid Sadik. CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts. arXiv:2601.04505, 2026. URL: <https://arxiv.org/abs/2601.04505>.
- Jesse Barkley, Rumi Loghmani, and Amir Barati Farimani. CADSmith: Multi-Agent CAD Generation with Programmatic Geometric Validation. arXiv:2603.26512, 2026. URL: <https://arxiv.org/abs/2603.26512>.
- CAD Arena. CAD Arena: Open Benchmark for AI-Generated Parametric CAD. 2026. URL: <https://cadarena.dev/>.
- Fan Cui, Hongyuan Hou, Zizhang Luo, Chenyun Yin, and Yun Liang. HWE-Bench: Benchmarking LLM Agents on Real-World Hardware Bug Repair Tasks. arXiv:2604.14709, 2026. URL: <https://arxiv.org/abs/2604.14709>.
- Epoch AI. CadEval. 2026. URL: <https://epoch.ai/benchmarks/cad-eval>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, 2023. URL: <https://arxiv.org/abs/2310.06770>.

Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL: <https://arxiv.org/abs/2409.17106>.

KiCad Project. KiCad Documentation. 2026. URL: <https://docs.kicad.org/>.

Jindong Li, Lianrong Chen, Bin Yang, Jiadong Zhu, Ying Wang, Yuzhe Ma, and Menglin Yang. PCB-Bench: Benchmarking LLMs for Printed Circuit Board Placement and Routing. In *International Conference on Learning Representations*, 2026. URL: <https://openreview.net/forum?id=Q5QLu7XTWx>.

Taiting Lu, Kaiyuan Lin, Yuxin Tian, Yubo Wang, Muchuan Wang, Sharique Khatri, Akshit Kartik, Yixi Wang, Amey Santosh Rane, Yida Wang, Yifan Yang, Yi-Chao Chen, Yincheng Jin, and Mahanth Gowda. OmniSch: A Multimodal PCB Schematic Benchmark For Structured Diagram Visual Reasoning. arXiv:2604.00270, 2026. URL: <https://arxiv.org/abs/2604.00270>.

Qinpei Luo, Ruichun Ma, Xinyu Zhang, and Lili Qiu. SchGen: PCB Schematic Generation with Semantic-Grounded Code Representations. OpenReview preprint, 2026. URL: <https://openreview.net/forum?id=TyWs6rWWHb>.

ngspice contributors. ngspice, the open-source SPICE circuit simulator. 2026. URL: <https://ngspice.sourceforge.io/>.

Weibo Qiu, Yinhao Xiao, and Runyu Pan. HWE-Bench: Can Language Models Perform Board-level Schematic Designs? arXiv:2603.18102, 2026. URL: <https://arxiv.org/abs/2603.18102>.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024. URL: <https://arxiv.org/abs/2404.07972>.

John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. ProgramBench: Can Language Models Rebuild Programs From Scratch? arXiv:2605.03546, 2026. URL: <https://arxiv.org/abs/2605.03546>.

Huanghao Zou, Peng Han, Emad Nazerian, and Alex Q. Huang. PCBSchemaGen: Constraint-Guided Schematic Design via LLM for Printed Circuit Boards. arXiv:2602.00510, 2026. URL: <https://arxiv.org/abs/2602.00510>.

A Artifact availability

The source repository contains the runner and harness code, task-pack loader and grader support, public documentation, package tests, and the task-pack manifest. Python wheels and source distributions exclude concrete task instances, reference projects, upstream snapshots, and control submissions. The Hugging Face task dataset contains runtime task packs, full grader task packs, prompts, references, upstream snapshots, control submissions, task metadata, Croissant metadata, and license notes.

The public task dataset is hosted at <https://huggingface.co/datasets/eda-bench-neurips-2026/eda-bench-tasks>. The reported results use task-pack revision c64b9169d6d8c0d73614e14c7050cf036aeb1559. The self-contained construction briefs are released at revision 55fb20f08ee499dd690b1991352ec2845d9e17fe. The public code release is hosted at <https://github.com/CAD-bench/eda-bench>.

Run records include task prompts, command lines, usage summaries, grader outputs, final projects, and generated artifacts. Storage-light campaigns retain final reports, final projects, grading artifacts, and byte-count summaries while omitting full Docker image uploads and raw model transcripts.

Table 6: Validation controls for score interpretation.

Control	Observed result
Frozen reference project	35/35 released references score 1.0
Structural fail canary	35/35 released fail canaries score at most 0.15, below the 0.75 release threshold
Generated mutation canaries	162 generated mutation canaries across released task families pass validation
Calibrated task-family checks	USB-C, RS-485, BQ24295, and M.2/PCIe references score 1.0 and their mutation canaries remain below threshold
Refdes-invariant equivalent board	USB-C reference with renamed connector refdes scores 1.0
Missing routed copper	USB-C and NanoUPDI route-removal controls score 0.0
Plausible route-width edit	USB-C route-width edit remains at least 0.85
Implausibly tiny traces	Manufacturability cap applies and USB-C tiny-trace control scores at most 0.35
Missing or unparsable project	Returns 0.0 and build failure

B Validation controls

C Failure-mode diagnostics

Table 7: Common electrical failure modes exposed by the evaluator in otherwise buildable submissions.

Failure mode	Observable symptom	Why it affects score
Split required external nets	Pins that should belong to the same external interface are assigned to disconnected copper regions or separate nets	Boundary stimuli cannot propagate through the declared interface, even when connector footprints are present
Isolated external I/O pads	Required pads are placed but not connected to routed copper or active circuitry	The board exposes a physical pad without realizing the electrical behavior required by the contract
Wrong pin or net mapping	External connector pins are swapped, mislabeled, or attached to semantically incorrect nets	The board may appear visually plausible while driving or observing the wrong external signal
Missing active realization	Components required for level shifting, power-path behavior, transceiver behavior, or control logic are absent or not connected	Passive connectivity alone cannot implement the task-required circuit behavior
Weak active-device power integrity	Active devices are present but lack required power, ground, or enable connectivity	The simulated or inferred behavior cannot be attributed to a realizable powered circuit
Implausible high-speed geometry	Differential or high-speed-like interface families are missing, weakly routed, or routed with implausible geometry	Dense carrier and compute tasks require routed interface families with plausible physical realization

D Baseline run records

The May 6, 2026 result table regrades saved final projects without rerunning model harnesses. Source runs were: `codex__gpt-5.5-web-low_20260505T200208Z_2719115`, `codex__gpt-5.5-web-medium_20260506T010715Z_2812175`, `codex__gpt-5.5-web-medium_20260506T010715Z_2812182`, `codex__gpt-5.5-web-medium_20260506T032737Z_2884928`, `codex__gpt-5.5-web-medium_20260506T010716Z_2812190`, `codex__gpt-5.5-web-high_20260505T200208Z_2719113`, `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884940`, `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884938`, `codex__gpt-5.5-web-xhigh_`

20260506T032737Z_2884943, codex__gpt-5.5-web-xhigh_20260506T034126Z_2891808,
pi__gemini-3.1-pro-preview-web-high_20260505T184850Z_2697190, and
pi__deepseek__deepseek-v4-pro-web-high_20260505T200208Z_2719116.

The release’s regrade entry point accepts one or more source-run identifiers, uses the released task manifest, downloads the saved `submission/final_project` directories, and runs the local evaluator. Passing the four Codex GPT-5.5 web medium identifiers above reproduces that aggregate row.

E Cost and compute accounting

The completed GPT-5.5 web runs used storage-light run records. The combined Codex GPT-5.5 low/medium/high/xhigh campaign uploaded about 185.6 MiB. Across all six final harnesses, the total upload was about 193.1 MiB.

Table 8: Recorded Codex GPT-5.5 web campaign accounting.

Harness	Estimated OpenAI API USD	Estimated Codex credits	HF records
Codex GPT-5.5 web low	38.22	955.41	48.9 MB
Codex GPT-5.5 web medium	84.40	2110.05	49.5 MB
Codex GPT-5.5 web high	49.39	1234.64	27.6 MB
Codex GPT-5.5 web xhigh	182.75	4568.64	68.6 MB

The estimates are computed from recorded token usage, current provider pricing, and the Codex token-based rate card. They make leaderboard costs and storage requirements directly auditable.

F License and redistribution

EDA Bench separates runtime prompts from full task packs with references, snapshots, controls, and grader contracts. Released task packs include source-backed materials with redistributable prompts, references, grader contracts, controls, and license notes. This structure preserves reproducibility while respecting the redistribution terms of upstream task materials.